

AwaazIQ

Workstream C — photo checks and fake-complaint defence · [change log](#)

Everything changed on branch `abhay-hanchate-c3` (Member 3), what each change does, how it was tested, and the measured accuracy of the team's pothole model. Written so the other workstreams can review, merge and build on it.

7 / 7

features C1–C7 built

81.8%

model accuracy on 88
held-out real photos

86%

of real potholes caught
(recall)

264

tests passing across 4
suites

Date: 15 September 2026 · Base: `origin/village-priority-system` merged into `abhay-hanchate-c3` · Status: working tree changes, **not yet committed**.

1. Summary

- **Photos are now taken live in the app** (no gallery), with GPS and time recorded at the moment of capture and a 5-frame burst.
- **Every photo is checked on upload**: taken near the village, not a copy of another report's photo, not edited with software, not a photo of a screen, and what the pothole/crack model sees in it.
- **Corroboration counts people, not wordings** (C2).
- **Results reach officers**: asset evidence keys, a photo evidence section in the asset panel, and a new **Photo Lab** page for testing any photo.
- **No complaint is ever rejected**. Checks only move confidence (never below half) and flag photos for review, per the plan's ground rule 5.

2. Features built

ID	Feature	What was done	Status
C1	Camera-only capture, GPS and time at capture	New camera modal using <code>getUserMedia</code> + Image Capture API; photo capped at 2560 px; GPS watched while open and read at the shutter press; 5 burst frames. The form no longer accepts image files (PDF letters only). Server stores capture method, time, GPS, accuracy and distance to the village; flags > 5 km, stale capture time, no GPS, plain file upload.	done
C2	Count distinct accounts, not wording	<code>recompute.reporter_key()</code> : a signed-in report counts as its account; anonymous/seeded reports fall back to distinct wording. Used by clusters, assets and the cluster reports endpoint, so the numbers agree.	done

C3	Duplicate-photo check	Perceptual hashes (<code>imagehash</code> <code>phash</code> + <code>dhash</code>) stored per photo; a match reused by another account or village is flagged <code>reused_photo</code> ; the same person resubmitting is only noted.	done
C4	Edit detection	Editing software in EXIF (Photoshop, Snapseed, PicsArt, screenshot...) is flagged for review. Error-level analysis gives a score and heat map for the officer. It could not separate edited from genuine photos in testing, so it is advisory only and never lowers trust.	done · advisory
C5	Pothole model as road evidence	Team YOLOv8 model runs on every photo (crack + pothole), boxes drawn on an annotated copy, results stored; rolled up to <code>evidence["photo_defect"]</code> on assets and clusters. A road complaint whose photo shows no damage is noted.	done
C6	Building/bridge damage grade	Grade crack (1/3) < partial (2/3) < collapse (1) from the photo's crack detections and damage extent plus catastrophic wording in English, Hindi, Marathi and Hinglish. Emergency candidate only for a bridge or building. A collapse is never marked "confirmed by photo". Rolled up to <code>evidence["emergency_damage"]</code> for B2.	done
C7	Screen-replay check	Moiré detection from spectral peaks in colour and brightness channels; burst liveness: identical frames (still image fed through a virtual camera) are flagged.	done

How the checks combine

Each flag lowers a photo's **authenticity** multiplicatively (floor 0.2). Verdict: *verified*, *partly verified* or *needs officer review*. The report's confidence becomes `confidence × (0.7 + 0.3 × worst authenticity)` , never below half, and a verified live photo that shows the damage lifts it. Stored on the report as `photo_trust` and `review_flags` .

3. Pothole model — accuracy and details

Item	Value
Model	YOLOv8 (m-size), 25.86 M parameters, Ultralytics 8.4.7, trained at 1024 px; classes <code>crack</code> , <code>pothole</code>
File	<code>models/pothole_best.pt</code> · SHA-256 <code>1d1f3fd0...1731</code> · 150 MB (git-ignored)
Checkpoint's own validation	Precision 0.759 · recall 0.708 · mAP50 0.773 · mAP50-95 0.468 (its training data source, so optimistic)
Weights changed?	No. The received folder was missing one 2-value optimizer buffer (used only to resume training); restored as zeros and re-zipped. All model weights are byte-identical.
AwaazIQ inference settings	1024 px · confidence ≥ 0.25 · ignore boxes under 0.5% of the frame · test-time augmentation on

Image-level accuracy ("does this photo show a pothole?")

Test set	n	Setting	Accuracy	Precision	Recall	F1	ROC-AUC
Held-out real photos (Wikimedia: 25 countries + Indian roads; never used for tuning)	88	default	81.8%	78.4%	78.4%	0.78	0.907
		AwaazIQ	81.8%	74.4%	86.5%	0.80	0.924
Tuning set (used to choose settings; optimistic)	64	default	71.9%	52.0%	68.4%	0.59	0.844
		AwaazIQ	84.4%	71.0%	79.0%	0.75	0.883
tarooi/pothole-detection test (224 px thumbnails)	72	default	52.8%	100%	26.1%	0.41	0.711
		AwaazIQ	66.7%	96.0%	50.0%	0.66	0.746

Held-out set: 37 clear potholes, 46 pothole-free roads (44 in Kerala, Karnataka, Gujarat, Tamil Nadu), 5 hard negatives. Labels checked by eye; ambiguous photos excluded. AwaazIQ settings caught 32 of 37 potholes with 11 false alarms.

Other measurements

Measurement	Result
Box-level, Roboflow "Potholes Detection" test (100 images, 269 boxes)	mAP50 0.311 default → 0.342 with TTA (its labels also box patched repairs, so this understates the model)
Crack class on damaged-bridge photos	Fired on 1 of 8 — a weak proxy for structures
Moiré (C7), fresh sample	0 of 50 genuine photos flagged; 15 of 25 simulated screen recaptures flagged
Error-level analysis (C4), fresh sample	6/50 genuine vs 3/25 spliced flagged — no discrimination, hence advisory
Speed (laptop CPU)	0.26 s/image at 640 px, 0.62 s at 1024 px; full photo check ≈ 2.9 s

Where it fails. Misses water-filled or debris-filled potholes, very small or distant ones and low-resolution photos. False alarms on dirt roads, busy junctions, painted road markings and natural river "potholes". Treat every result as a signal for an officer.

4. Files changed

New files

File	Purpose
backend/photo_checks.py	All checks C1, C3–C7, combining logic and confidence adjustment; tunables with reasons
backend/routes_photo_checks.py	Runs checks for a filed report; Photo Lab, model, report-check, annotated and ELA endpoints
backend/test_photo_checks.py	57 tests (unit + API, on a throw-away database)
backend/test_data/photos/	Two attributed Wikimedia test photos (CC BY-SA) + <code>ATtribution.md</code>
frontend/js/camera-capture.js	Live camera modal: photo, burst, GPS and time at capture
frontend/js/photo-checks.js , css/photo-checks.css	Shared check card used by receipt, asset panel and Photo Lab
frontend/photo-lab.html , js/photo-lab.js , css/photo-lab.css	Photo Lab page with model accuracy and honest limits
models/README.md , evaluation.json , eval_manifest.json , evaluate_pothole_model.py	Model card, measured accuracy, labelled test list and a script to re-run the evaluation
changes-abhayhanchate-c3.pdf	This document

Modified files

File	Change
backend/models.py	New <code>PhotoCheck</code> table; <code>citizen_request.photo_trust</code> , <code>review_flags</code>
backend/migrate.py , create_tables.py	Add the two new columns to existing databases
backend/routes_citizen_report.py	Runs photo checks after attachments are saved; receipt returns <code>photo_checks</code> and <code>photo_trust</code>
backend/routes_villages.py	<code>GET /assets/{id}</code> returns the asset's photo checks
backend/routes_dashboard.py	Cluster "distinct reporters" uses the same account-based count as scoring (C2)
backend/main.py	Registers the photo-check router

<code>intelligence/recompute.py</code>	C2 <code>reporter_key</code> ; loads photo checks; adds <code>photo_verification</code> , <code>photo_defect</code> , <code>emergency_damage</code> evidence. Scores are unchanged; the 9-term breakdown still sums.
<code>intelligence/test_intelligence.py</code>	Tests for C2 and the evidence roll-up
<code>frontend/report.html</code> , <code>js/report.js</code>	Camera-first evidence section, capture metadata and bursts sent, photo checks on the receipt; Leaflet integrity hashes fixed
<code>frontend/index.html</code> , <code>js/app.js</code>	Photo evidence section in the asset panel; Photo Lab link; version tags bumped to <code>?v=photo1</code>
<code>.gitignore</code> , <code>README.md</code>	Ignore model weights and eval downloads; README brought up to date

New or changed API

Endpoint	Purpose
POST <code>/citizen-report</code> (extended)	Accepts <code>capture_meta</code> (JSON, one entry per attachment) and <code>burst_<i>i</i></code> frames
POST <code>/photo-checks/analyze</code>	Run every check on one photo; stores nothing
GET <code>/photo-checks/model</code>	Model details and evaluation
GET <code>/photo-checks/report/{id}</code>	Checks for one report, without coordinates or hashes
GET <code>/report-attachment/{id}/annotated</code> , <code>/ela</code>	Photo with boxes; error-level image

5. Testing

Suite	Result
<code>backend/test_photo_checks.py</code> (new)	57/57
<code>python -m intelligence.test_intelligence</code>	96/96
<code>backend/test_citizen_report_endpoint.py</code>	65/65
<code>pipeline/test_pipeline.py</code>	46/46
Browser run (headless Chromium, fake camera + GPS)	Photo Lab, camera capture and receipt worked end to end; no console errors after the Leaflet fix. Test data removed afterwards.

6. Other things found

- **Fixed:** the Leaflet integrity hashes in `report.html` were wrong, so the browser blocked Leaflet and the exact-location pin map was broken. Hashes checked against cdnjs.
- **Fixed:** when the text pipeline cannot classify a description, photo checks now use the category of the department the citizen chose.
- **Not fixed (Workstream B):** a report with no category becomes a "Water point" asset in `_build_assets`.

- **Setup:** the database was rebuilt locally from the loaders; `load_lgd_hierarchy.py` needs `py7zr` installed.

7. Before merging

- **Model file:** `models/pothole_best.pt` is not in git (150 MB). Share it separately; without it photo checks still run, minus C5.
- **Database:** run `python create_tables.py` in `backend/` to add the new columns.
- **Packages:** `ultralytics`, `torch`, `imagehash`, `scipy`.
- **Not yet tested on a real phone**, including a phone photographing a screen. Chromium's fake camera degraded frames, so the in-browser detection could not be judged there.
- **For Workstream B:** the evidence keys `photo_defect` and `emergency_damage` are ready for B1/B2; no scoring weights were changed.